

Question 1: Heuristic design and admissibility

For steps, the heuristic used was to count how many characters in the current node's state deviated from the goal state. Consider the pseudocode below:

IF cost is "steps":

 heuristic <- 0

 FOR each pair of corresponding characters in node.state and goal:

 IF the characters are different:

 heuristic <- heuristic + 1

 RETURN heuristic

The unavoidable remaining cost this heuristic lower-bounds is the minimum number of actions needed to reach the goal. It assumes that all intermediate actions are legal, even if they aren't, i.e., the resulting word from the action does not exist in the dictionary.

As a result, the heuristic cannot overestimate the true optimal remaining cost. In the best-case scenario, every action leads to a character in the correct spot in the word. Every character that the node and goal states differ must eventually be changed. This heuristic counts the number of changes needed, but in reality, there might be extra intermediate steps needed. Therefore, the heuristic cannot overestimate the true optimal remaining cost.

Now for Scrabble, the heuristic is calculated by adding up the Scrabble value of the goal character in every incorrect position. Consider its pseudocode below:

IF cost is "scrabble":

 heuristic <- 0

 FOR each pair of corresponding character in node.state and goal:

 IF the characters are different:

 heuristic <- heuristic + ScrabbleCost(goal character)

 RETURN heuristic

This heuristic lower-bounds the minimum Scrabble cost needed to change the incorrect letters in the node state to the correct goal letters. Like steps, it also assumes that changing the incorrect character to the correct goal character is a legal move.

Consequently, this heuristic can also not overestimate the true optimal remaining cost. Since every incorrect character must be changed to the correct goal character, at

minimum, the Scrabble value of the correct goal character must be applied to the optimal cost. This heuristic assumes this minimum cost, but there are typically other actions required. Thus, it can only ever underestimate the minimum optimal cost or get it exactly right.

Finally, for frequency, the heuristic is very similar to the steps heuristic, except it also adds the rarity value of the goal state from the dictionary to the final heuristic. See the pseudocode below:

```
IF cost is "frequency":
```

```
    IF node.state is the goal:
```

```
        RETURN 0
```

```
    heuristic <- 0
```

```
    FOR each pair of corresponding characters in node.state and goal:
```

```
        IF the characters are different:
```

```
            heuristic <- heuristic + 1
```

```
    RETURN heuristic + rarity(goal)
```

This heuristic lower-bounds the optimal remaining cost by summing up the number of incorrect characters that need to be changed and the rarity or frequency value of the goal word in the dictionary. If the state is already the goal, then the cost is 0. Like the other two heuristics, it also assumes that changing the incorrect characters to the correct characters are legal moves and that the frequency cost of the goal state is unavoidable. Just counting the number of incorrect positions is admissible, as I discussed in the steps heuristic. Furthermore, since we assume that the frequency cost of the goal state is unavoidable, this cost must be applied to the true optimal remaining cost if we haven't already reached the goal state. Therefore, the true optimal remaining cost, at minimum, must include the number of incorrect characters in the current state, and the frequency cost of the goal state. As a result, this heuristic is admissible.

Question 2: Does the heuristic actually help?

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY -> ASK	Steps	5	773	5	16	97.93%
WHY -> ASK	Scrabble	9	765	9	13	98.30%

WHY -> ASK	Frequency	38.855	808	38.855	634	21.54%
BEAR -> DUCK	Steps	4	691	4	6	91.13%
BEAR -> DUCK	Scrabble	11	2981	11	6	99.80%
BEAR -> DUCK	Frequency	38.240	1084	38.240	302	72.14%

1. UCS and A* always returned the same optimal cost.
2. For both word pairs, the Scrabble cost measure got largest percentage reduction in expanded nodes from A*.
3. For both word pairs, the heuristic was least informative for the frequency cost measure.
4. The difference can be explained by how the heuristics estimate the remaining cost. For the steps cost, the number of mismatched letters closely represents the remaining cost which enables the large improvement of using A* over UCS. The Scrabble heuristic, on the other hand, considers the different Scrabble values for each goal state letter that is currently mismatched. The Scrabble values consider the rarity of each letter in the English dictionary. This is a more informative heuristic since if the letter is rarer, it is harder to find intermediate words to get from the current state to the goal state, likely explaining the large improvement in using the heuristic. The frequency heuristic, on the other hand, is like the steps the heuristic but adds a constant. Since it doesn't reflect the frequency costs for different intermediate states/words, it doesn't seem to provide much context, especially since the number of mismatched characters doesn't give much of an indication of the remaining cost when $g(n)$ is measured in frequency cost.

Question 3: Reversibility and direction-dependent costs

1. Yes, for the steps cost measure, the optimal cost from W1 to W2 must always equal the optimal cost from W2 to W1. This is because the cost of changing a character is always 1. Therefore, every path has the same cost when reversed, so the optimal cost must always be equal in both directions.
2. No, for the Scrabble cost measure, the optimal cost from W1 to W2 does not always equal the optimal cost from W2 to W1. The Scrabble value of the letter changed in the resulting word is used when calculating the cost. Therefore, the cost can change based on direction. For instance, consider the words in the dictionary: cent and tent. When going from cent to tent, we are changing C to T. Thus, we use T's Scrabble value which is 1. On the other hand, when we go from tent to cent, we

change T to C, so we use C's Scrabble value, which is 3. Therefore, the optimal cost is different based on which direction we go.

3. No, for the frequency cost measure, the optimal cost from W_1 to W_2 does not always equal the optimal cost from W_2 to W_1 . The frequency cost is calculated based on the resulting word, not the initial word. Therefore, reversing the path doesn't guarantee the same optimal cost. For instance, consider cent and tent again. The frequency of tent is 7.69 while the frequency of cent is 7.46. Therefore, when we go from cent to tent, the cost is 7.69 (we use tent's frequency cost because it is the resulting word) and when we go from tent to cent, the cost is 7.46 (we use cent's frequency cost because it is the resulting word). Since the costs are not the same, we cannot get the same optimal cost when reversing the paths.

Question 4: LLM heuristic audit

1. The model I'm using the Gemini Flash 3.6.

Here was the provided prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristics $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By "more informative," we mean that the proposed heuristic should at be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Here was it's proposed heuristic:

Proposed Heuristic $h_1(n)$

Let:

- n be the current word (state) and g be the goal word.
- $d(n, g)$ be the Hamming distance between n and g (the baseline heuristic $h_0(n)$).
- $\text{rarity}(w) \geq 0$ be the non-negative rarity cost of entering word w .
- $R_{\min} = \min_{w \in \mathcal{D} \setminus \{n\}} \text{rarity}(w)$ be the global minimum rarity value across all valid dictionary words (excluding the current word n).
- $R_{\min}^{(k)}$ be the sum of the k smallest distinct non-negative rarity values among all dictionary words $w \in \mathcal{D} \setminus \{n\}$.

We define the rarity-aware heuristic $h_1(n)$ as:

$$h_1(n) = \begin{cases} 0 & \text{if } n = g \\ d(n, g) + R_{\min}^{(d(n, g))} & \text{if } n \neq g \end{cases}$$

*(Simplified variant if calculating multi-word minimum sums is computationally expensive:
 $h_1(n) = d(n, g) \cdot (1 + R_{\min})$).*

To summarize the proposed heuristic, it essentially calculates the number of mismatches characters (call it d) and adds to d the d lowest rarities. See the pseudocode below:

IF cost is "ai":

IF node.state is the goal:

RETURN 0

$d \leftarrow 0$

FOR each pair of corresponding characters in node.state and goal:

IF the characters are different:

$d \leftarrow d + 1$

sorted_rarities $\leftarrow$ sorted(rarities)

rarity_sum $\leftarrow 0$

FOR $i \leftarrow 0$ to d :

rarity_sum $\leftarrow$ rarity_sum + sorted_rarities(i)

RETURN $d + \text{rarity_sum}$

2. Now, this heuristic is in fact admissible. It lower-bounds the optimal remaining cost by summing up the number of incorrect characters that need to be changed and the d smallest rarity values in the dictionary where d is the number of incorrect characters. If the state is already the goal, then the cost is 0. Like the other heuristics, it assumes that changing the incorrect characters to correct characters are legal actions, even if the intermediate words don't appear in the dictionary. Just counting the number of incorrect positions is admissible, which was determined in the steps heuristic. Furthermore, because reaching the goal requires taking at least d steps, at every step, there must be another rarity accumulated from the dictionary. Since we only ever use the d smallest rarities in the dictionary, we cannot overestimate the true optimal path cost, because the summed-up rarities can only ever exceed the minimums. Therefore, this heuristic is admissible.
3. Benchmarks:

Word Pair	Cost Measure	UCS Cost	UCS Nodes Expanded	A* Cost	A* Nodes Expanded	Reduction
WHY -> ASK	AI Frequency	38.855	808	38.855	748	7.42%
BEAR -> DUCK	AI Frequency	38.240	1084	38.240	681	37.18%

Comparing this frequency heuristic with the one from question 2, it seems that the expanded node percentage reduction for both word pairs is worse in the ai-generated heuristic. This might mean the d -minimum rarities lack context that the original implementation has. However, it does provide some reduction, not rendering it completely useless, but I don't see using it over the previous one.